Robot Design Speech

Look it's a Giraffe! (Team 69255)

Opening

Hello, we're Look it's a Giraffe! and today we will explain how we designed our robot. Our strategy is to attempt missions that got us lots of points and were relatively easy. We looked at all the different missions and ranked them by two things: they needed to be able to get us lots of points, so we have a higher chance of doing better. The other way was how easy it is to achieve the mission, so we can go fast and get more missions done.

We did the missions in that order because we wanted to get from one side to the other just once to save time.

When building and coding the robot we used bot-fluencers, conference sessions, the SPIKE app and build instructions.

Choosing and building our robot

We looked at 4 different designs: HummerOne, Comp Bot, GummyBears / Droid Bot M and StarterBot. We scored them on six criteria, all about how the robot performs in a match: archetype match to proven competitive designs, attachment swap speed at base, squaring (flat back and sides), stability under load, maneuverability in tight spaces, and drive repeatability (for PID code).

After we finished building HummerOne we noticed the front wheels would slide and create friction, which made it harder to move straight. We changed the front wheels to two mini ball casters, which helped cause less friction, which helped the robot move straight.

Our attachments

The forklift (Mission 9). Jack started building the forklift from Coach Zain. Ella and Amelia continued building it. We made a couple of mistakes, like when Sylvie

moved up the gear bar. A problem Ella found was that certain things were built on the wrong side. Sylvie and others fixed this by going back in the instructions and rebuilding certain parts.

The lifting arm (Mission 2). The design it originated from was from a bot-fluencer. We added a seed bucket because it couldn't bring the seeds back without one. We also added axle pins because it didn't stay on properly. The robot didn't line up with the mission. To fix this, we added what we call the masher. The masher forces and mashes the robot into place. It is good when it does its job, but it's not very consistent.

The drone survey attachment (Mission 1). The design was inspired by Team TurtleBots. We made one similar to theirs. We added some smaller curved pieces so it wouldn't get caught on things or on the mission. The attachment used to be wiggly and loose. Our solution was to add extra points for it to attach to itself, as well as some other pieces. It is good at flipping open the lidar map, but it can't get the drone there.

Our code and sensors

We use 2 colour sensors and 1 gyro sensor. The colour sensors sense the colours on the mat. We use them for squaring.

The reason we added a second colour sensor is that squaring is when one sensor sees the colour black, it will turn until the other sensor sees black. It will then wiggle till straight. It drives crooked without squaring. First, it drives up to the line diagonally and senses the black line. Then it turns until the other sensor senses the black line. Finally, it wiggles until it's straight.

Our code uses a block called Gyro Move. Every 0.1 seconds the robot checks the yaw angle, and if it is off it corrects itself. Kp decides how it corrects itself over a short period of time. Ki decides how it corrects itself over a long period of time.

For turning we use yaw angle instead of motor degrees because yaw is more reliable. We also increase the speed of the turn depending on how big the turn is.

The robot starts slowly, speeds up and slows at the end. That makes it so that the robot doesn't go farther than we want it to.

We changed the distance block from degrees to cm because degrees are not as easy to measure as cm.

Testing and improving

We wanted to know if we had a good Kp and Ki. We tested Kp and Ki 15 times. The result was that the Kp was already good. When we tested it at 1 the average error was 26 mm, and when we tested 3 the error was 13. Our original Kp of 2 was under 1 mm error. We tested Ki at 0.5, 1 and 2. We found that changing it from 2 to 1 made us more accurate by 5 mm. It worked very well.

We also wanted to know if we could complete our missions in the time limit. We tested it by doing the missions with a stopwatch repeatedly. So we added blocks with a timer so the robot would carry on instead of stopping and wasting tokens. We wasted less precision tokens and got more points in the end. We still need to work on some funky code.